

C++ 프로그래밍 (STL 알고리즘)

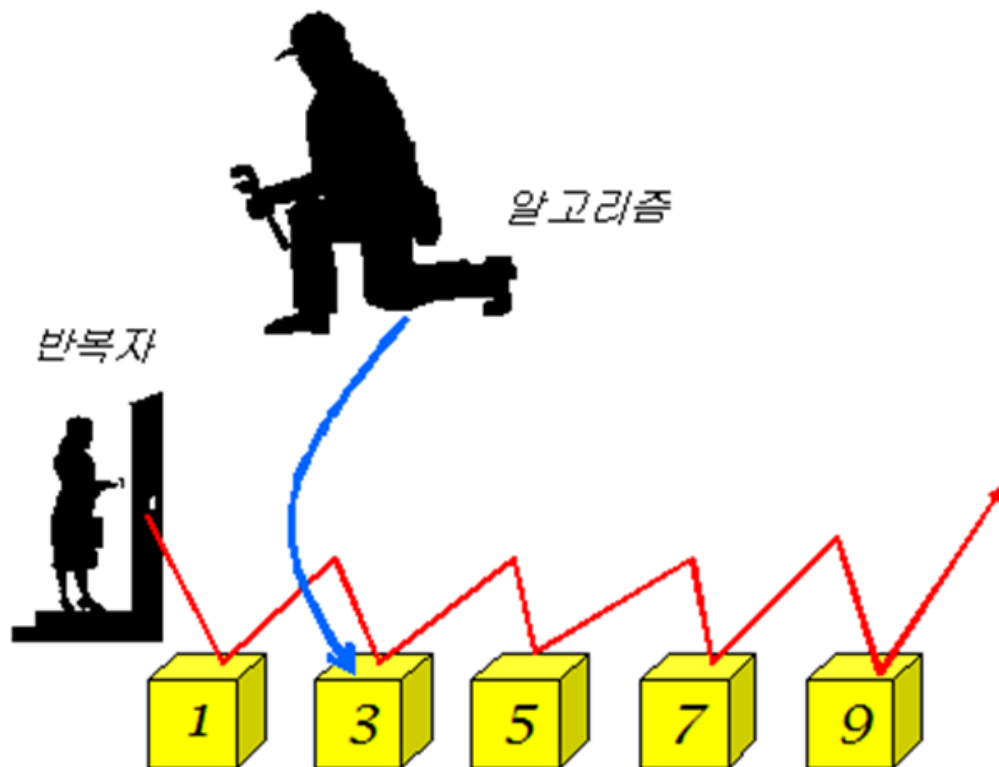
14주차

목차

- STL 알고리즘
- 반복자
- 탐색 알고리즘
- 비교 알고리즘
- 초기화 알고리즘
- 수치 알고리즘
- 함수 객체

STL 알고리즘

- STL 알고리즘
 - STL 기반의 탐색(searching), 정렬(sorting), 계수(counting) 알고리즘



STL 알고리즘 (계속)

- STL 알고리즘의 분류

STL 알고리즘

불변경 알고리즘:
데이터를 변경하지 않음



변경 알고리즘:
데이터가 변경됨



STL 알고리즘 (계속)

- 불변경 알고리즘

분류	알고리즘	설명
계수 알고리즘	<code>count()</code>	주어진 값과 일치하는 요소들의 개수를 센다.
	<code>count_if()</code>	주어진 조건에 맞는 요소들의 개수를 센다.
탐색 알고리즘	<code>search()</code>	주어진 값과 일치하는 첫번째 요소를 반환한다.
	<code>search_n()</code>	주어진 값과 일치하는 n개의 요소를 반환한다.
	<code>find()</code>	주어진 값과 일치하는 첫번째 요소를 반환한다.
	<code>find_if()</code>	주어진 조건에 일치하는 첫번째 요소를 반환한다.
	<code>find_end()</code>	주어진 조건에 일치하는 마지막 요소를 반환한다.
	<code>binary_search()</code>	정렬된 컨테이너에 대하여 이진 탐색을 수행한다.
비교 알고리즘	<code>equal()</code>	두개의 요소가 같은지 비교한다.
	<code>mismatch()</code>	두개의 컨테이너를 비교하여서 일치하지 않는 첫번째 요소를 반환한다.
	<code>lexicographical_compare()</code>	두개의 순차 컨테이너를 비교하여서 사전적으로 어떤 컨테이너가 작은지를 반환한다.

STL 알고리즘 (계속)

- 변경 알고리즘

분류	알고리즘	설명
초기화 알고리즘	fill()	지정된 범위의 모든 요소를 지정된 값으로 채운다.
	generate()	지정된 함수의 반환값을 할당한다.
변경 알고리즘	for_each()	지정된 범위의 모든 요소에 대하여 연산을 수행한다.
	transform()	지정된 범위의 모든 요소에 대하여 함수를 적용한다.
복사 알고리즘	copy()	하나의 구간을 다른 구간으로 복사한다.
삭제 알고리즘	remove()	지정된 구간에서 지정된 값을 가지는 요소들을 삭제한다.
	unique()	구간에서 중복된 요소들을 삭제한다.
대치 알고리즘	replace()	지정된 구간에서 요소가 지정된 값과 일치하면 대치값으로 바꾼다.
정렬 알고리즘	sort()	지정된 정렬 기준에 따라서 구간의 요소들을 정렬한다.
분할 알고리즘	partition()	지정된 구간의 요소들을 조건에 따라서 두개의 집합으로 나눈다.

반복자

- 컨테이너에서 다음 요소를 가리키기 위한 ++연산자
- 컨테이너에서 이전 요소를 가리키기 위한 - 연산자
- 두개의 반복자가 같은 요소를 가리키고 있는 지를 확인하기 위한 ==와 != 연산자
- 반복자가 가리키는 요소의 값을 추출하기 위한 역참조 연산자 *
- v.begin()함수는 컨테이너 v에서 첫 번째 요소를 반환한다.
- v.end()함수는 컨테이너 v에서 마지막 요소를 지났는지를 나타내는 값을 반환한다.

반복자 (계속)

- 반복자의 종류
 - 전향 반복자(forward iterator) : ++ 연산자만 가능하다.
 - 양방향 반복자(bidirectional iterator) : ++ 연산자와 -- 연산자가 가능하다.
 - 무작위 접근 반복자(random access iterator) : ++ 연산자와 -- 연산자, [] 연산자가 가능하다.

반복자 (계속)

- 예제

```
int main()
{
    vector<int> vec;           // 정수형 벡터 생성
    for(int i=0;i<10;i++)
        vec.push_back(i);

    vector<int>::iterator it;   // 반복자 객체 생성
    for(it=vec.begin(); it != vec.end() ; it++)    // 컨테이너의 모든 요소를 출력
        cout << *it << " ";
    cout << endl;

    for(it=vec.begin(); it != vec.end() ; it++)    // 컨테이너의 모든 요소를 -1로 설정
        *it = 0;

    for(it=vec.begin(); it != vec.end() ; it++)    // 컨테이너의 모든 요소를 출력
        cout << *it << " ";
    cout << endl;
    return 0;
}
```

실행 결과

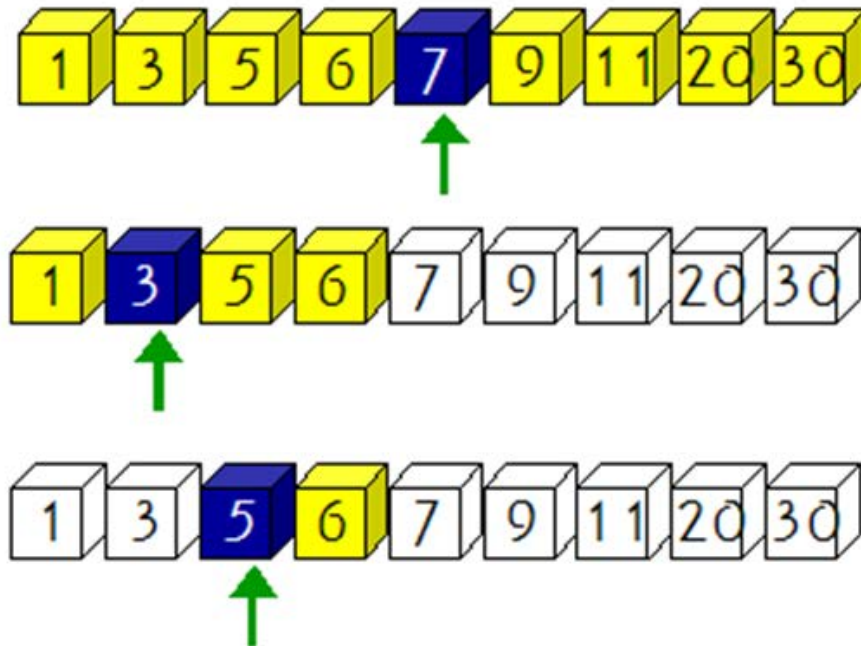
0 1 2 3 4 5 6 7 8 9

0 0 0 0 0 0 0 0 0 0

계속하려면 아무 키나 누르십시오 . . .

탐색 알고리즘

- 이진 탐색
 - 정렬된 리스트에서 만약 찾고자 하는 원소가 중간 원소보다 크면 찾고자 하는 원소는 뒷부분에 있고 반대이면 앞부분에 있다.



탐색 알고리즘 (계속)

- 예제

```
#include <iostream>
#include <vector>
#include <algorithm>
#include "myheader.h"
using namespace std;

int main()
{
    const int wanted = 6;
    int values[9] = { 1, 3, 5, 5, 5, 8, 11, 20, 30 };
    vector<int> vec(&values[0], &values[9]);
    vector<int>::iterator it;

    print(vec);
    bool isInt = binary_search(vec.begin(), vec.end(), wanted);
```

```
    if(isInt == true )
        cout << wanted << "을 찾았음 " << endl;
    else {
        it = lower_bound(vec.begin(), vec.end(), wanted);
        vec.insert(it, wanted);
    }
    print(vec);

    return 0;
}
```

실행 결과

(1 3 5 5 5 8 11 20 30)

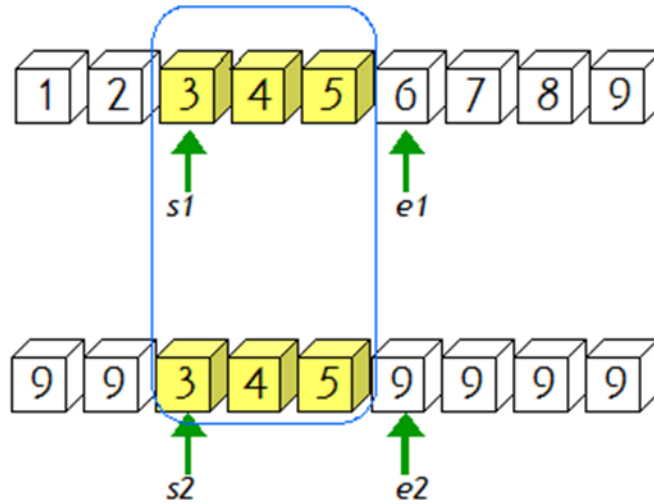
(1 3 5 5 5 6 8 11 20 30)

계속하려면 아무 키나 누르십시오 . . .

비교 알고리즘

- equal()

```
bool  
equal(ForwIter s1, ForwIter e1, ForwIter s2, [UnaryPred func]);
```



비교 알고리즘 (계속)

- 예제

```
int main()
{
    int values1[9] = { 1, 2, 3, 4, 5, 6, 7, 8, 9 };
    int values2[9] = { 9, 9, 3, 4, 5, 9, 9, 9, 9 };
    vector<int> vec1(&values1[0], &values1[9]);
    vector<int> vec2(&values2[0], &values2[9]);
    print(vec1);
    print(vec2);
    bool isEqual = equal(vec1.begin()+2, vec1.begin()+5, vec2.begin()+2);

    if(isEqual == true )
        cout << "두개의 구간이 동일함" << endl;

    return 0;
}
```

실행 결과

(1 2 3 4 5 6 7 8 9)

(9 9 3 4 5 9 9 9 9)

두개의 구간이 동일함

계속하려면 아무 키나 누르십시오 . . .

초기화 알고리즘

- fill() 함수
 - 어떤 연속성을 띤 자료구조(벡터나 배열 같은)의 시작점부터 연속된 범위를 어떤 값이나 객체로 모두 지정하고 싶을 때 사용하는 함수

초기화 알고리즘 (계속)

- 예제

```
int main()
{
    vector<int> v1;
    for( int i = 0; i < 10; i++ ) {
        v1.push_back( i );
    }
    cout << "fill() 이전의 값 ";
    print(v1);
    fill( v1.begin(), v1.end(), 0);

    cout << "fill() 이후의 값 ";
    print(v1);
    return 0;
}
```

실행 결과

fill() 이전의 값 (0 1 2 3 4 5 6 7 8 9)

fill() 이후의 값 (0 0 0 0 0 0 0 0 0 0)

계속하려면 아무 키나 누르십시오 . . .

수치 알고리즘

- copy()함수
 - 벡터에 있는 내용을 복사 할 때 사용
- for_each()함수
 - 범위 내에 원소들에 대해 함수를 실행 시킴
- transform()함수
 - 반복자 구간에 대해 함수 객체를 적용한 후 그 결과를 다른 구간에 복사
- remove()함수
 - 반복자들과 삭제할 원소 값을 받으며 해당 원소를 찾아 삭제
- sort()함수
 - 범위에 있는 인자를 오름/내림 차순으로 정렬

수치 알고리즘 (계속)

- copy() 예제

```
// copy와 reverse의 사용 예
```

```
#include <iostream>
```

```
#include <algorithm>
```

```
#include <string>
```

```
#include <vector>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    string names[5] = {"사과", "배", "키위", "레몬", "포도"};
```

```
    vector<string> fruits(5);
```

```
    vector<string>::iterator it;
```

```
    copy(&names[0], &names[5], fruits.begin());
```

```
    reverse(fruits.begin(), fruits.end());
```

```
    for (it = fruits.begin(); it != fruits.end(); ++it)
```

```
        cout << *it << " ";
```

```
    cout << endl;
```

```
}
```

실행 결과

포도 레몬 키위 배 사과

계속하려면 아무 키나 누르십시오 . . .

수치 알고리즘 (계속)

- for_each() 예제

```
// 각 요소에 대하여 호출되는 함수  
void display(int element)  
{  
    cout << element << ' ';  
}  
int main()  
{  
    vector<int> vec;  
    for(int i=0; i<10; i++)  
        vec.push_back(i);  
  
    for_each (vec.begin(), vec.end(), display);  
    cout << endl;  
    return 0;  
}
```

실행 결과

0 1 2 3 4 5 6 7 8 9

계속하려면 아무 키나 누르십시오 . . .

수치 알고리즘 (계속)

- transform() 예제

```
// 각 요소에 대하여 호출되는 함수
```

```
int increment(int element)
```

```
{
```

```
    return ++element;
```

```
}
```

```
int main()
```

```
{
```

```
    vector<int> vec;
```

```
    vector<int> result(10);
```

```
    for(int i=0; i<10; i++)
```

```
        vec.push_back(i);
```

```
    print(vec);
```

```
    transform (vec.begin(), vec.end(), result.begin(), increment);
```

```
    print(result);
```

```
    return 0;
```

```
}
```

실행 결과

```
( 0 1 2 3 4 5 6 7 8 9 )
```

```
( 1 2 3 4 5 6 7 8 9 10 )
```

계속하려면 아무 키나 누르십시오 . . .

수치 알고리즘 (계속)

- remove() 예제

```
int main()
{
    string names[5] = {"사과", "배", "키위", "레몬", "포도"};
    vector<string> fruits(5);
    copy(&names[0], &names[5], fruits.begin());
    print(fruits, "삭제 전\n");
    vector<string>::iterator it;
    it = remove(fruits.begin(), fruits.end(), "레몬");
    print(fruits, "remove() 후\n");
    fruits.erase(it, fruits.end());
    print(fruits, "erase() 후\n");
    return 0;
}
```

실행 결과

삭제 전

(사과 배 키위 레몬 포도)

remove() 후

(사과 배 키위 포도 포도)

erase() 후

(사과 배 키위 포도)

수치 알고리즘 (계속)

- sort() 예제

```
#include <iostream>
#include <algorithm>
#include <vector>
#include "myheader.h"
using namespace std;

int main()
{
    int value[10]={ 82, 25, 26, 7, 67, 55, 31, 19, 99 };
```

```
    vector<int> v1(&value[0],&value[10]);
    print(v1,"초기 리스트\n");
    sort (v1.begin(), v1.end());
    print(v1, "sort() 적용후 리스트\n");
```

```
    vector<int> v2(&value[0],&value[10]);
    stable_sort (v2.begin(), v2.end());
    print(v2, "stable_sort() 적용후 리스트\n");
```

```
    vector<int> v3(&value[0],&value[10]);
    partial_sort (v3.begin(), v3.begin()+3, v3.end());
    print(v3, "partial_sort() 적용후 리스트\n");
```

```
    vector<int> v4(&value[0],&value[10]);
    nth_element(v4.begin(), v4.begin()+5, v4.end());
    print(v4, "nth_element() 적용후 리스트\n");
```

```
    return 0;
```

```
}
```

실행 결과

초기 리스트

(82 25 26 7 67 55 31 19 99 0)

sort() 적용후 리스트

(0 7 19 25 26 31 55 67 82 99)

stable_sort() 적용후 리스트

(0 7 19 25 26 31 55 67 82 99)

partial_sort() 적용후 리스트

(0 7 19 82 67 55 31 26 99 25)

nth_element() 적용후 리스트

(0 7 19 25 26 31 55 67 82 99)